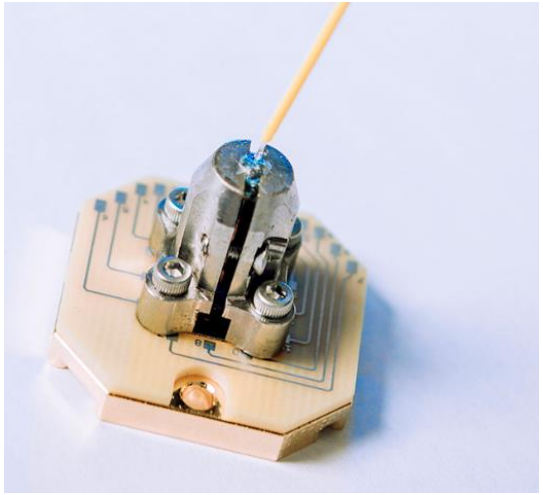


QUANDELA

Generating Mario levels with Photonic Quantum Machine Learning

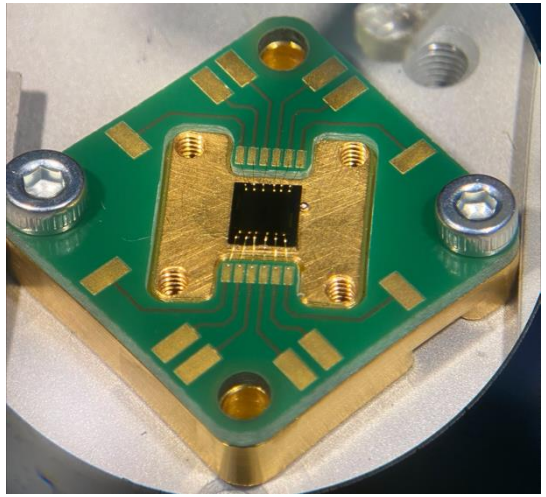
www.quandela.com

From single photons to a quantum processor



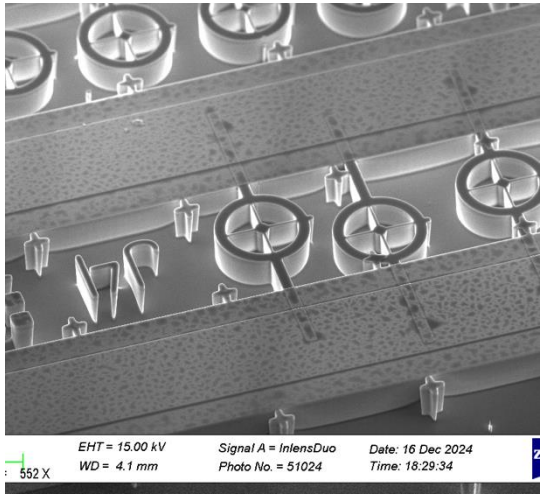
01

Single-photon source



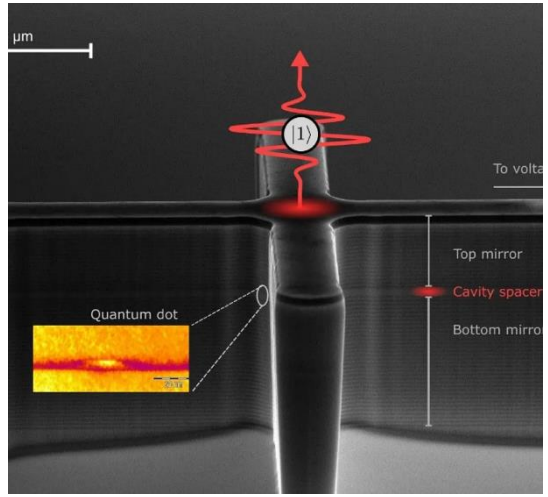
02

Photonic chip



03

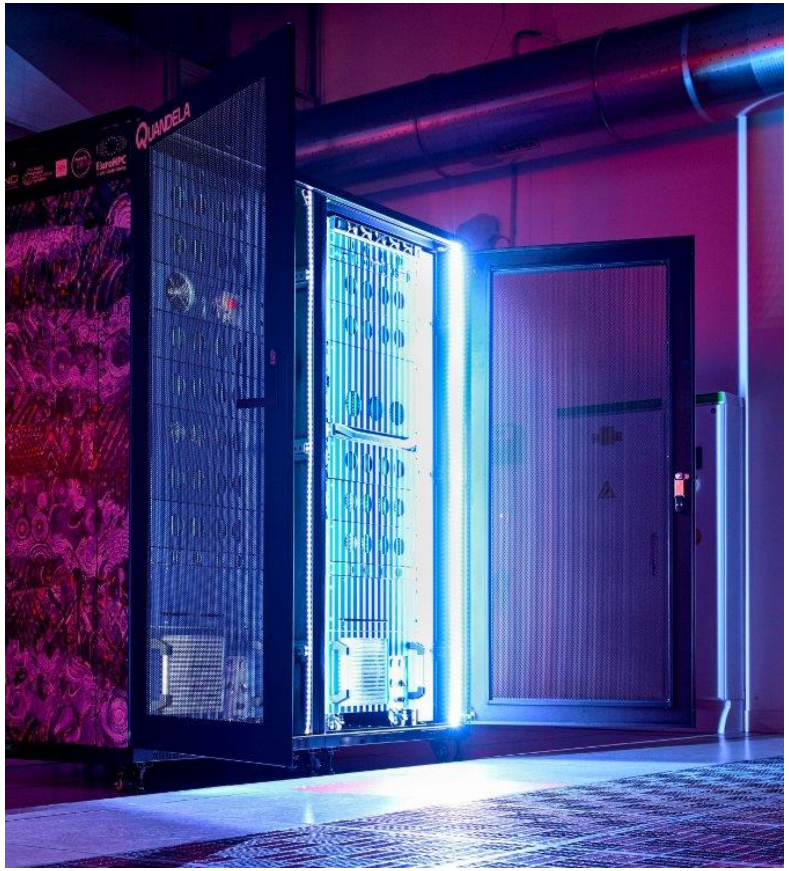
Integrated photonics



04

Optical control

What can we explore with MosaiQ?



01

Hybrid HPC

Prepare workflows combining classical and quantum computing.

02

Quantum ML

Explore feature maps and reservoir computing workflows.

03

Real hardware

Compare simulation with measured photon samples.

04

Photonic effects

Study interference, losses and imperfect photons.

Photonic quantum computing

How photons become a computing resource

01

Working Principles

CONFIDENTIAL - COPYRIGHT QUANDELA



Discover MerLin by reproducing a paper

Quantum Reservoir Computing for Level Generation

Demonstration

MerLin 0.4

QUANDELA

The course follows the scientific workflow

MerLin › **Paper** › **Reservoir** › **Generation** › **Debrief**

Setup and overview

Task and data

Photonics and readout

Temperature and metrics

What the results show

Guiding thread

Understand **what stays fixed**, what learns, and what the metrics support.

QUANDELA

MerLin makes photonics compatible with PyTorch



MerLin is open source and supports QML workflow development, simulation and execution without requiring linear optics expertise.

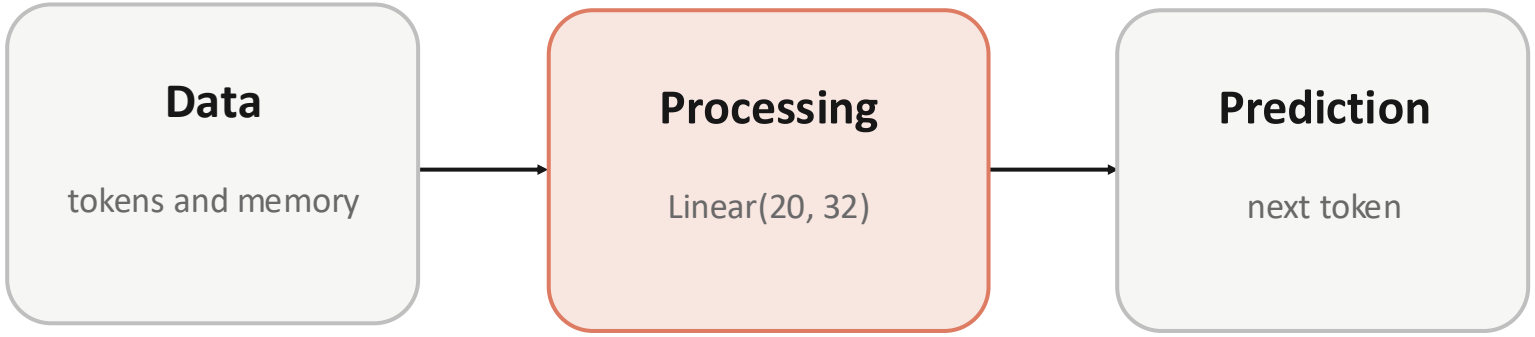
A QuantumLayer brings the photonic circuit, data encoding and measurement together in a differentiable module.

Input	classical tensor
Circuit	CircuitBuilder or Perceval
Measurement	probabilities, expectations or amplitudes
Output	tensor ready for a classical head

QUANDELA

A quantum layer

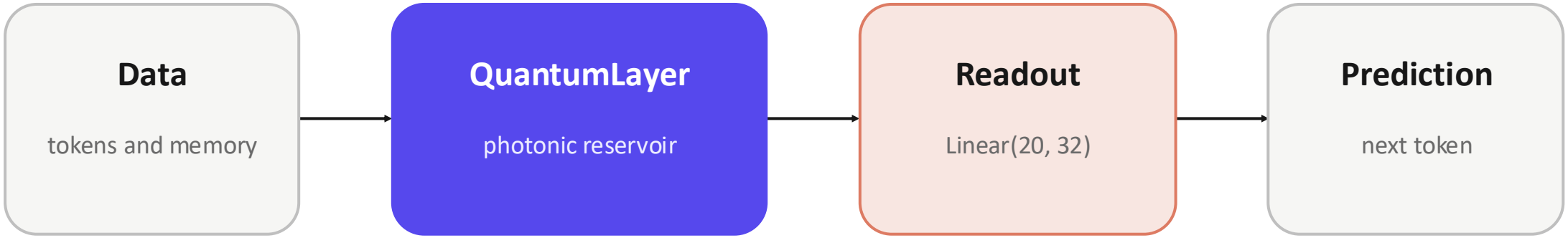
fits into the middle of a classical learning pipeline



QUANDELA

A quantum layer

fits into the middle of a classical learning pipeline



In this workshop, the reservoir stays fixed. Adam optimizes only the readout.

QUANDELA

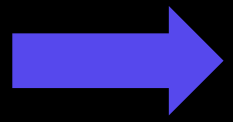
https://arxiv.org/abs/2505.13287

Generate playable variants

Training

1 Mario level

The model learns transitions from a single sequence.



Generation

multiple variants

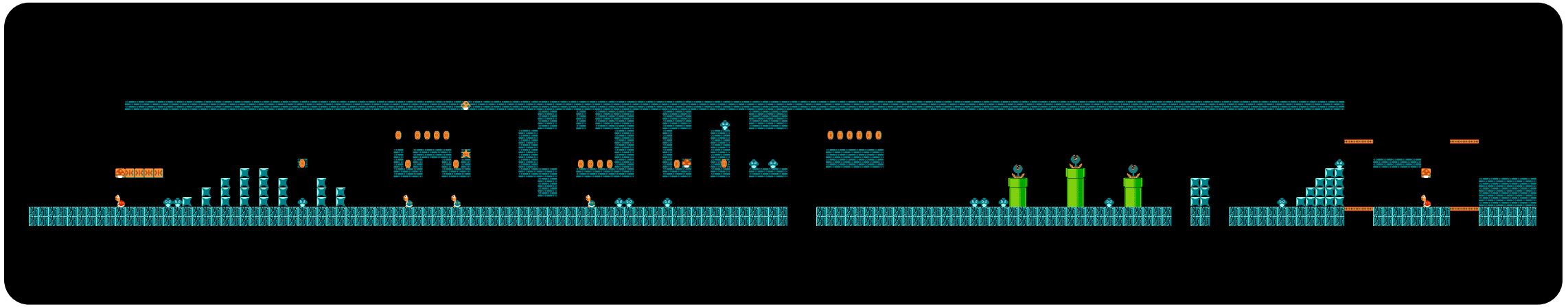
New variants without multiplying gameplay errors

Two competing criteria: novelty and transition integrity

The MerLin notebook makes the paper executable and lets us test its modelling choices.

QUANDELA

One level becomes a sequence



16 px

width of one column

157

tokens in the level

32

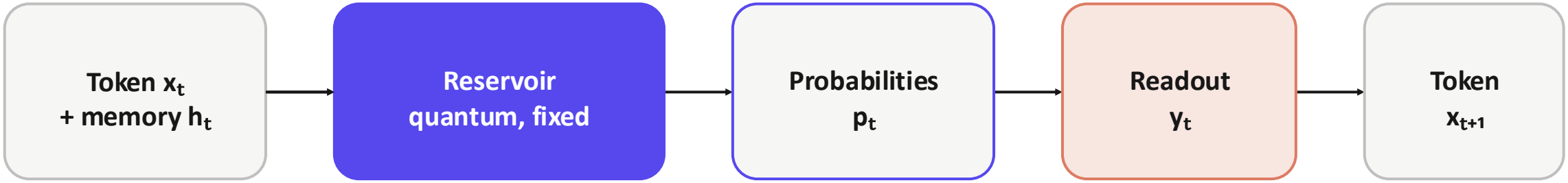
unique features

156

next-token pairs

QUANDELA

The quantum reservoir computing loop



$$h_{t+1} = (1 - \varepsilon)h_t + \varepsilon p_t, with\ \varepsilon = 0.3$$

$$x_{t+1} \sim Categorical(softmax(y_t / T))$$

The memory is classical. The reservoir computation is quantum.

QUANDELA

Four objects to know how to read

01	**CircuitBuilder**	describes the modes, encoding and interferometers
02	**QuantumLayer**	runs the circuit as a PyTorch module
03	**MeasurementStrategy**	selects the probabilities and the UNBUNCHED space
04	**nn.Linear**	converts 20 probabilities into 32 logits

QUANDELA

The circuit remains readable

```
builder = ML.CircuitBuilder(n_modes=6)
builder.add_entangling_layer(trainable=False)
builder.add_angle_encoding(
    modes=list(range(6)),
    name="qrc_input", scale=np.pi,
)
builder.add_entangling_layer(trainable=False)

layer = ML.QuantumLayer(
    builder=builder, n_photons=3,
    measurement_strategy=ML.MeasurementStrategy.probs(
        computation_space=ML.ComputationSpace.UNBUNCHED))
```

6 modes

input dimension

angle encoding

six values to six phases

3 photons

measured dimension: 20

probabilities

normalized output for the readout